Pergunta Vaga: Faça uma calculadora de IMC em Python

Com certeza! O Índice de Massa Corporal (IMC) é uma conta simples, mas muito útil para uma avaliação inicial de saúde.

O cálculo segue a fórmula:

$$IMC = \frac{\text{peso}}{\text{altura}^2}$$
Aqui está um script em Python que solicita os dados do usuário, calcula o índice e já retorna a classificação baseada na tabela da OMS:

Python

```python
def calcular_imc():
    print("--- Calculadora de IMC ---")

    try:
        # Solicita os dados ao usuário
        peso = float(input("Digite seu peso (kg): ").replace(',', '.'))
        altura = float(input("Digite sua altura (m): ").replace(',', '.'))

        # Realiza o cálculo
        imc = peso / (altura ** 2)

        print(f"\nSeu IMC é: {imc:.2f}")

        # Classificação do resultado
        if imc < 18.5:
            print("Classificação: Abaixo do peso")
        elif 18.5 <= imc < 25:
```

```
            print("Classificação: Peso normal (parabéns!)")
        elif 25 <= imc < 30:
            print("Classificação: Sobrepeso")
        elif 30 <= imc < 35:
            print("Classificação: Obesidade Grau I")
        elif 35 <= imc < 40:
            print("Classificação: Obesidade Grau II")
        else:
            print("Classificação: Obesidade Grau III (Mórbida)")

    except ValueError:
        print("Erro: Por favor, digite apenas números válidos para peso e altura.")

# Executa a função
if __name__ == "__main__":
    calcular_imc()
```

Destaques do código:

- **`replace(',', '.')`**: Usei isso para que o programa não quebre caso o usuário digite a altura com vírgula (ex: 1,75) em vez de ponto.
- **`f"{imc:.2f}"`**: Formata o número para exibir apenas duas casas decimais, deixando o resultado mais limpo.
- **Tratamento de Erros**: O bloco `try/except` evita que o programa feche caso alguém digite letras onde deveriam estar números.

Dica de amigo: Lembre-se que o IMC é uma métrica geral. Ele não diferencia gordura de massa muscular, então, se você for um atleta de elite, o resultado pode ser meio enganoso!

Gostaria que eu explicasse como transformar esse script em uma interface gráfica ou como salvar os resultados em um arquivo de texto?

Pergunta Mestre

Olá! Como seu mentor técnico e engenheiro de software, fico feliz em ver que você está priorizando a **modularização** e a **validação de dados** desde o início. No desenvolvimento de software profissional, não basta que o código funcione; ele precisa ser resiliente e fácil de manter.

Abaixo, apresento a solução estruturada para o seu módulo de saúde.

---

1. Lógica Matemática

O Índice de Massa Corporal (IMC) é uma medida internacional usada para calcular se uma pessoa está no peso ideal. A fórmula matemática é:

$$IMC = \frac{\text{peso}}{\text{altura}^2}$$
Onde:

- **Peso** é medido em quilogramas (kg).
- **Altura** é medida em metros (m).

A lógica de classificação segue os intervalos padrão da Organização Mundial da Saúde (OMS), utilizando estruturas condicionais (`if/elif/else`) para determinar em qual categoria o resultado se encaixa.

---

2. Implementação em Python (PEP 8)

Este script foi escrito seguindo as recomendações da **PEP 8**, com nomes de funções em `snake_case`, constantes e tipagem de dados (*type hints*) para melhor legibilidade.

Python

```python
"""
Módulo de Saúde: Calculadora de IMC
Autor: Mentor Técnico / Engenheiro de Software
"""

def obter_entrada_float(mensagem: str) -> float:
```

```python
    """
    Solicita uma entrada do usuário e valida se é um número real positivo.
    """
    while True:
        try:
            valor_str = input(mensagem).replace(',', '.')
            valor = float(valor_str)
            if valor <= 0:
                print("Erro: O valor deve ser maior que zero.")
                continue
            return valor
        except ValueError:
            print("Erro: Entrada inválida. Digite apenas números.")

def calcular_imc(peso: float, altura: float) -> float:
    """
    Realiza o cálculo do IMC: peso / (altura^2).
    """
    return peso / (altura ** 2)

def classificar_imc(imc: float) -> str:
    """
    Retorna a classificação do IMC baseada nos padrões da OMS.
    """
    if imc < 18.5:
        return "Abaixo do peso"
    elif 18.5 <= imc < 25:
        return "Peso ideal (Parabéns!)"
    elif 25 <= imc < 30:
        return "Sobrepeso"
    elif 30 <= imc < 35:
        return "Obesidade Grau I"
    elif 35 <= imc < 40:
        return "Obesidade Grau II"
    else:
        return "Obesidade Grau III (Mórbida)"

def executar_calculadora():
    """
    Função principal que coordena o fluxo do programa.
    """
    print("=== Módulo de Saúde: Cálculo de IMC ===")

    peso = obter_entrada_float("Digite seu peso em kg (ex: 75.5): ")
    altura = obter_entrada_float("Digite sua altura em metros (ex: 1.75): ")
```

```
    # Processamento
    valor_imc = calcular_imc(peso, altura)
    resultado_classificacao = classificar_imc(valor_imc)

    # Saída formatada
    print("\n--- Resultado ---")
    print(f"Seu IMC: {valor_imc:.2f}")
    print(f"Classificação: {resultado_classificacao}")
    print("----------------\n")

if __name__ == "__main__":
    executar_calculadora()
```

---

3. Como executar o script

Para rodar o código, certifique-se de ter o Python 3 instalado em sua máquina e siga estes passos:

1. Copie o código acima e salve-o em um arquivo chamado `calculadora_imc.py`.
2. Abra o seu terminal ou prompt de comando.
3. Navegue até a pasta onde o arquivo foi salvo.
4. Execute o comando:
5. Bash

```
python calculadora_imc.py
```

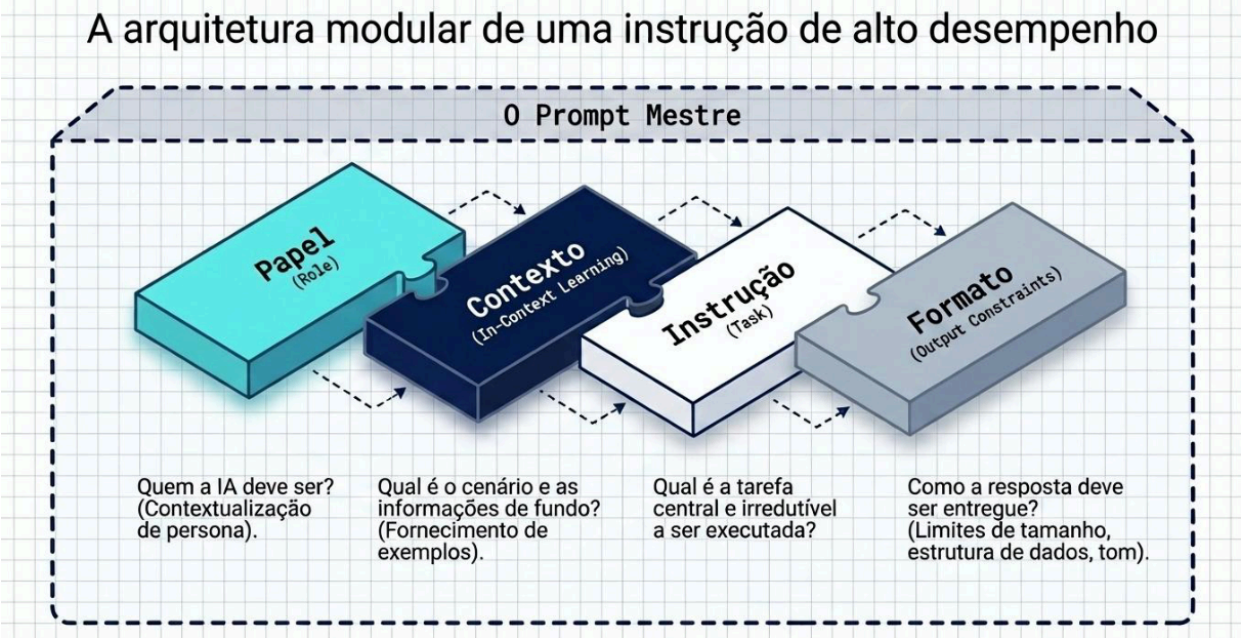
6.
7.

Exemplo de Interação:

Plaintext

```
=== Módulo de Saúde: Cálculo de IMC ===
Digite seu peso em kg (ex: 75.5): 82
Digite sua altura em metros (ex: 1.75): 1.80

--- Resultado ---
Seu IMC: 25.31
Classificação: Sobrepeso
----------------
```

Dica de Mentor: Note que usei o bloco `if __name__ == "__main__":`. Isso é uma boa prática fundamental em Python; ele garante que a calculadora só será executada se você rodar o arquivo diretamente. Se você importar este módulo em outro projeto no futuro, ele não sairá pedindo inputs automaticamente.

Gostaria que eu demonstrasse como criar **testes unitários** para essas funções usando o `pytest` para garantir que o cálculo nunca falhe?